

Chương 3

Số học máy tính

Bộ tính số học và ALU



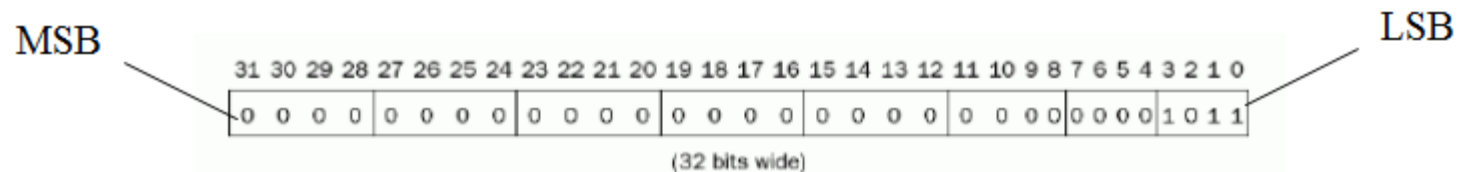
Biểu diễn các số có dấu và không dấu

Các số có dấu và không có dấu

$$\begin{aligned} & (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)_{\text{ten}} \\ &= (1 \times 8) + (0 \times 4) + (1 \times 2) + (1 \times 1)_{\text{ten}} \\ &= 8 + 0 + 2 + 1_{\text{ten}} \\ &= 11_{\text{ten}} \end{aligned}$$

```
0000 0000 0000 0000 0000 0000 0000 0000two = 0ten
0000 0000 0000 0000 0000 0000 0000 0001two = 1ten
0000 0000 0000 0000 0000 0000 0000 0010two = 2ten
...
1111 1111 1111 1111 1111 1111 1111 1101two = 4,294,967,293ten
1111 1111 1111 1111 1111 1111 1111 1110two = 4,294,967,294ten
1111 1111 1111 1111 1111 1111 1111 1111two = 4,294,967,295ten
```

Biểu diễn số 13d (1011b) bằng số nhị phân 32 bit



Số nguyên không dấu (n-bit unsigned Integer)

A gồm n con số nhị phân (n digit) được biểu diễn như sau:

$$A \Rightarrow a_{n-1} a_{n-2} \dots a_1 a_0$$

Tính ra giá trị của A trong hệ đếm 10 :

$$A = a_{n-1} 2^{n-1} + a_{n-2} 2^{n-2} + \dots + a_1 2^1 + a_0 2^0 = \sum_{i=0}^{i=n-1} a_i 2^i$$

Số nguyên có dấu (n-bit signed Integer)

$$A \Rightarrow a_{n-1} a_{n-2} \dots a_1 a_0$$

- **Biểu diễn kiểu đánh dấu cho số có trị tuyệt đối (sign-magnitude representation)**

$$\text{nếu } |A| \geq 0 \rightarrow a_{n-1} = 0 \Rightarrow A = \sum_{i=0}^{i=n-2} a_i 2^i$$

$$A \Rightarrow 0 a_{n-2} \dots a_1 a_0$$

$$\text{nếu } |A| < 0 \rightarrow a_{n-1} = 1 \Rightarrow A = - \sum_{i=0}^{i=n-2} a_i 2^i$$

$$A \Rightarrow 1 a_{n-2} \dots a_1 a_0$$

Thí dụ: +18 = 0 0010010 - 18 = 1 0010010

Số bù

- Biểu diễn kiểu bù 2 (two-complement representation)

$$\begin{aligned} + \text{Số bù 1:} \quad A &\Rightarrow 0 \ a_{n-2} \ a_{n-3} \dots a_1 \ a_0 \\ -A &\Rightarrow 1 \ \overline{a_{n-2}} \ \overline{a_{n-3}} \dots \overline{a_1} \ \overline{a_0} \end{aligned}$$

+ Số bù 2:

$$\begin{aligned} A &\Rightarrow -2^{n-1} a_{n-1} + 2^{n-2} a_{n-2} + \dots + 2^1 a_1 + 2^0 a_0 = \\ &= -2^{n-1} a_{n-1} + \sum_{k=0}^{n-2} 2^k a_k \end{aligned}$$

Bù 2

starting value	00100100 = +36
step1: reverse the bits (1's complement)	11011011
step 2: add 1 to the value from step 1	+ 1
sum = 2's complement representation	11011100 = -36

Sum of an integer and its 2's complement must be zero:

$00100100 + 11011100 = 00000000$ (8-bit sum) $\Rightarrow$ Ignore Carry

Another way to obtain the 2's complement:

Start at the least significant 1

Leave all the 0s to its right unchanged

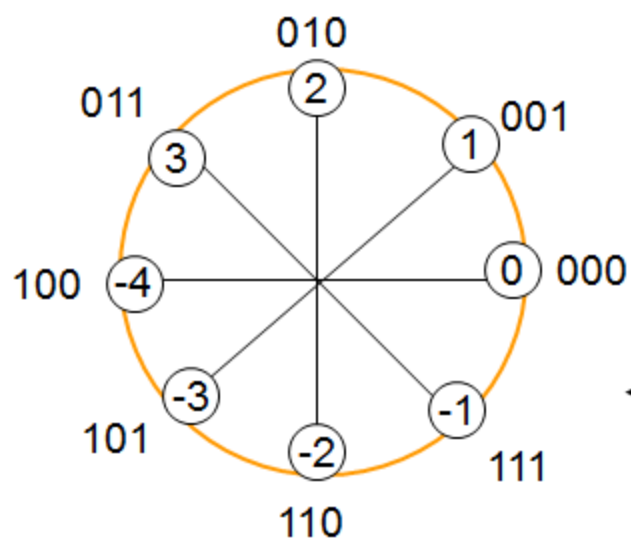
Complement all the bits to its left

Binary Value

= 00100 **1** 00 least significant 1

2's Complement

= **11011** **1** 00



dải số $A : -2^{n-1} \div 2^{n-1} - 1$

← 3 bit : dải số (-4 ÷ +3)

Bù 2 = Bù 1 + 1 ; chỉ số 1 số không !

- Thí dụ: dùng 8 bit +50d = 0011 0010 -70d = 1011 1010

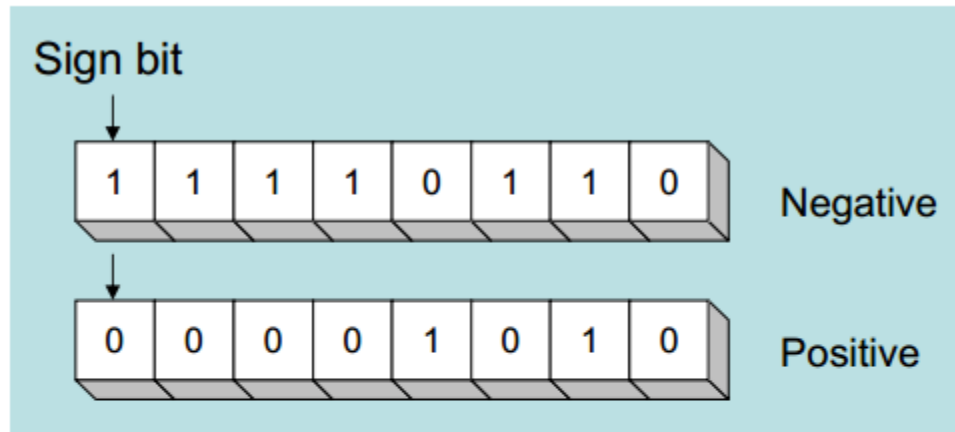
- Chuyển các số có độ dài bit khác nhau:

$$+18 = 00010010 \quad +18 = 00000000 \text{ (green) } 00010010 \text{ (red)}$$

$$-18 = 10010010 \quad -18 = 11111111 \text{ (green) } 10010010 \text{ (red)}$$

Bit dấu

- ❖ Highest bit indicates the sign
- ❖ 1 = negative
- ❖ 0 = positive



For Hexadecimal Numbers, check most significant digit

If highest digit is > 7 , then value is negative

Examples: 8A and C5 are negative bytes

B1C42A00 is a negative word (32-bit signed integer)

Bù 2 của số 16

❖ To form the two's complement of a hexadecimal

✧ Subtract each hexadecimal digit from 15

✧ Add 1

❖ Examples:

2's complement of **6A3D** = **95C2** + 1 = **95C3**

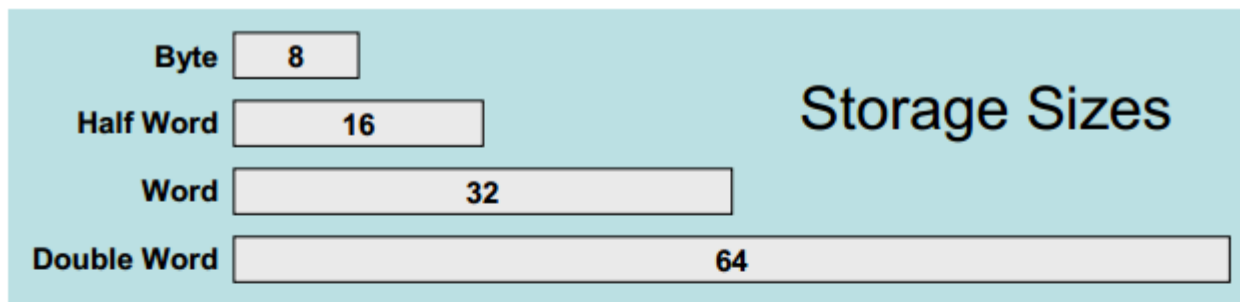
2's complement of **92F15AC0** = **6D0EA53F** + 1 = **6D0EA540**

2's complement of **FFFFFFFF** = **00000000** + 1 = **00000001**

❖ No need to convert hexadecimal to binary

- Chuyển đổi số nhị phân sang số hệ 10
- Chuyển đổi số hệ 10 không dấu sang số hệ nhị phân
- Hệ Hexa
- Chuyển đổi nhị phân sang hệ Hexa
- Chuyển đổi hệ Hexa sang hệ 10
- Chuyển đổi hệ 10 sang hệ Hexa

Kích cỡ số nguyên



Storage Type	Unsigned Range	Powers of 2
Byte	0 to 255	0 to ($2^8 - 1$)
Half Word	0 to 65,535	0 to ($2^{16} - 1$)
Word	0 to 4,294,967,295	0 to ($2^{32} - 1$)
Double Word	0 to 18,446,744,073,709,551,615	0 to ($2^{64} - 1$)

What is the largest 20-bit unsigned integer?

Answer: $2^{20} - 1 = 1,048,575$

Thực hiện các phép cộng và trừ các số nguyên

- Phép cộng: qui tắc nhị phân bình thường
- Phép trừ: cộng số bị trừ với số bù 2 của số trừ

Cộng nhị phân

- Bắt đầu từ bit ít ý nghĩa nhất (bên phải của dãy bit)
- Cộng từng cặp bit
- Thực hiện cộng bit nhớ (nếu có)

carry		1	1	1	1				
	0	0	1	1	0	1	1	0	(54)
+	0	0	0	1	1	1	0	1	(29)
	0	1	0	1	0	0	1	1	(83)
bit position:	7	6	5	4	3	2	1	0	

Trừ nhị phân

- ❖ When subtracting $A - B$, convert B to its 2's complement
- ❖ Add A to $(-B)$

borrow: 1 1 1		carry: 1 1 1 1
0 1 0 0 1 1 0 1		0 1 0 0 1 1 0 1
- 0 0 1 1 1 0 1 0	→	+ 1 1 0 0 0 1 1 0 (2's complement)
0 0 0 1 0 0 1 1		0 0 0 1 0 0 1 1 (same result)

- ❖ Final carry is ignored, because
 - ✧ Negative number is sign-extended with 1's
 - ✧ You can imagine infinite 1's to the left of a negative number
 - ✧ Adding the carry to the extended 1's produces extended zeros

Cộng số 16

- ❖ Start with the least significant hexadecimal digits
- ❖ Let Sum = summation of two hex digits
- ❖ If Sum is greater than or equal to 16
 - ✧ Sum = Sum – 16 and Carry = 1
- ❖ Example:

$$\begin{array}{rcccccccc} \text{carry:} & & & & 1 & 1 & & 1 \\ + & 1 & C & 3 & 7 & 2 & 8 & 6 & A \\ & 9 & 3 & 9 & 5 & E & 8 & 4 & B \\ \hline & A & F & C & D & 1 & 0 & B & 5 \end{array}$$

A + B = 10 + 11 = 21
Since $21 \geq 16$
Sum = $21 - 16 = 5$
Carry = 1

Trừ số 16

16 + 5 = 21

↓

Borrow: 1 1 1

B 1 4 F C 6 7 5	
- 8 3 9 E A 2 4 7	
2 D B 1 2 4 2 E	

➡

Carry: 1 1 1 1 1

B 1 4 F C 6 7 5	
+ 7 C 6 1 5 D B 9	(2's complement)
2 D B 1 2 4 2 E	(same result)

- ❖ When a borrow is required from the digit to the left, then
Add 16 (decimal) to the current digit's value
- ❖ Last Carry is ignored

Khoảng biểu diễn của số nguyên có dấu

For n -bit signed integers: Range is -2^{n-1} to $(2^{n-1} - 1)$

Positive range: 0 to $2^{n-1} - 1$

Negative range: -2^{n-1} to -1

Storage Type	Unsigned Range	Powers of 2
Byte	-128 to +127	-2^7 to $(2^7 - 1)$
Half Word	-32,768 to +32,767	-2^{15} to $(2^{15} - 1)$
Word	-2,147,483,648 to +2,147,483,647	-2^{31} to $(2^{31} - 1)$
Double Word	-9,223,372,036,854,775,808 to +9,223,372,036,854,775,807	-2^{63} to $(2^{63} - 1)$

Practice: What is the range of signed values that may be stored in 20 bits?

Carry and Overflow

❖ Carry is important when ...

- ✧ Adding or subtracting **unsigned integers**
- ✧ Indicates that the **unsigned sum** is out of range
- ✧ Either < 0 or $>$ maximum unsigned n -bit value

❖ Overflow is important when ...

- ✧ Adding or subtracting **signed integers**
- ✧ Indicates that the **signed sum** is out of range

❖ Overflow occurs when

- ✧ Adding two positive numbers and the sum is negative
- ✧ Adding two negative numbers and the sum is positive
- ✧ Can happen because of the fixed number of sum bits

Carry and Overflow Examples

- ❖ We can have carry without overflow and vice-versa
- ❖ Four cases are possible (Examples are 8-bit numbers)

				1					
	0	0	0	0	1	1	1	1	15
+	0	0	0	0	1	0	0	0	8
	0	0	0	1	0	1	1	1	23
Carry = 0 Overflow = 0									

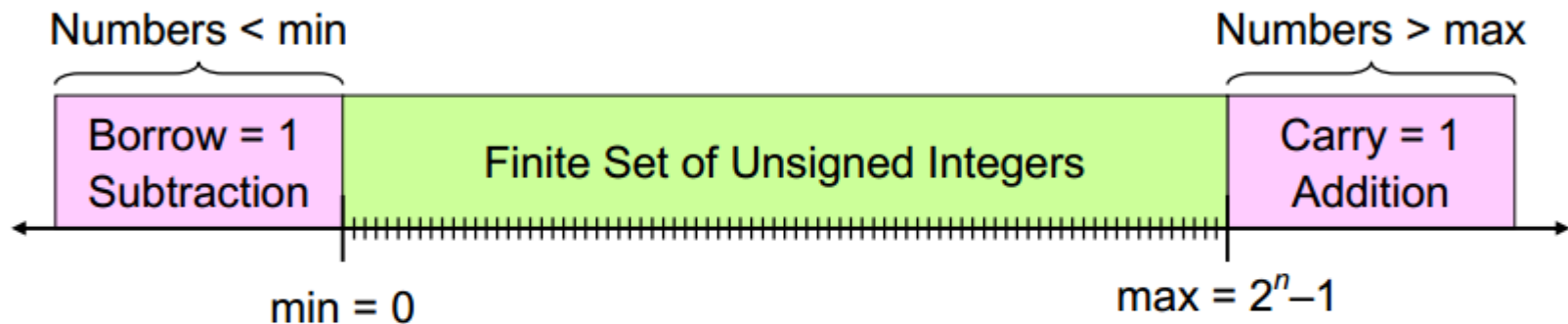
1	1	1	1	1					
	0	0	0	0	1	1	1	1	15
+	1	1	1	1	1	0	0	0	248 (-8)
	0	0	0	0	0	1	1	1	7
Carry = 1 Overflow = 0									

				1					
	0	1	0	0	1	1	1	1	79
+	0	1	0	0	0	0	0	0	64
	1	0	0	0	1	1	1	1	143 (-113)
Carry = 0 Overflow = 1									

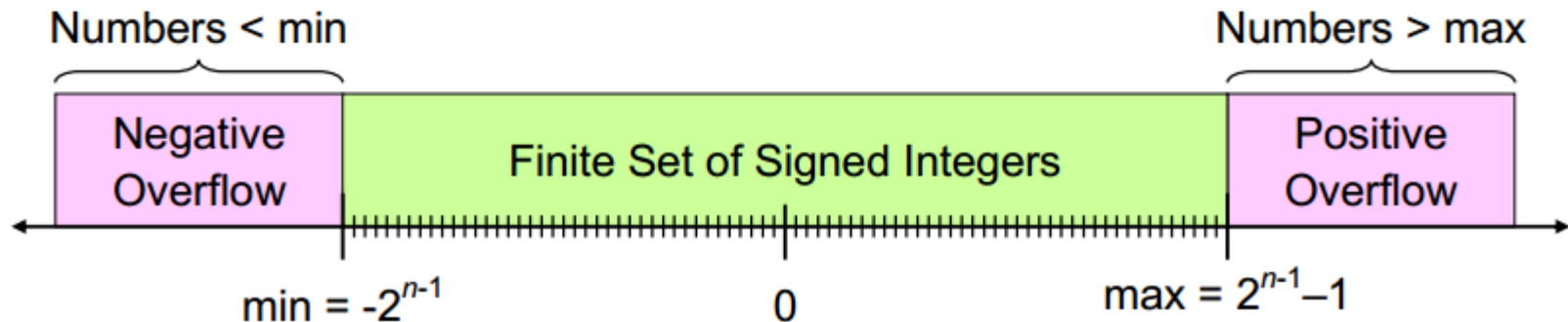
1				1		1			
	1	1	0	1	1	0	1	0	218 (-38)
+	1	0	0	1	1	1	0	1	157 (-99)
	0	1	1	1	0	1	1	1	119
Carry = 1 Overflow = 1									

Range, Carry, Borrow, and Overflow

❖ Unsigned Integers: n -bit representation



❖ Signed Integers: n -bit 2's complement representation



Character Storage

❖ Character sets

- ✧ Standard ASCII: 7-bit character codes (0 – 127)
- ✧ Extended ASCII: 8-bit character codes (0 – 255)
- ✧ Unicode: 16-bit character codes (0 – 65,535)
- ✧ Unicode standard represents a universal character set
 - Defines codes for characters used in all major languages
 - Used in Windows-XP: each character is encoded as 16 bits
- ✧ UTF-8: variable-length encoding used in HTML
 - Encodes all Unicode characters
 - Uses 1 byte for ASCII, but multiple bytes for other characters

❖ Null-terminated String

- ✧ Array of characters followed by a NULL character

Printable ASCII Codes

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
2	space	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

❖ Examples:

- ❖ ASCII code for space character = 20 (hex) = 32 (decimal)
- ❖ ASCII code for 'L' = 4C (hex) = 76 (decimal)
- ❖ ASCII code for 'a' = 61 (hex) = 97 (decimal)

Control Characters

- ❖ The first 32 characters of ASCII table are used for control
- ❖ Control character codes = 00 to 1F (hexadecimal)
 - ✧ Not shown in previous slide
- ❖ Examples of Control Characters
 - ✧ Character 0 is the **NULL** character $\Rightarrow$ used to terminate a string
 - ✧ Character 9 is the **Horizontal Tab (HT)** character
 - ✧ Character 0A (hex) = 10 (decimal) is the **Line Feed (LF)**
 - ✧ Character 0D (hex) = 13 (decimal) is the **Carriage Return (CR)**
 - ✧ The LF and CR characters are used together
 - They advance the cursor to the beginning of next line
- ❖ One control character appears at end of ASCII table
 - ✧ Character 7F (hex) is the **Delete (DEL)** character